# Resumen de estudio — SQL y MySQL

**Bases de datos relacionales · Solo contenido de estudio**

> Resumen reorganizado a partir del material de clase (¿Qué es MySQL?, Tipo de datos en MySQL, Constraints en SQL, Clase JOIN y la práctica DATABASE clinica_medica).
>
> La **sección 7 (GROUP BY) es extra**: no está en el material, pero es lo que usan los enunciados de la práctica; va con resultados verificados. Se conservan definiciones, clasificaciones, ejemplos, tablas y datos relevantes; se unificaron las repeticiones y cada concepto está explicado con palabras simples.

---

## Índice


1. ¿Qué es SQL?
2. Tipos de sentencias SQL: DDL, DML y DCL
3. Cláusulas, operadores y condiciones
4. Tipos de datos en MySQL
5. Constraints en SQL
6. JOIN: combinar tablas
7. Agregaciones y GROUP BY
8. Práctica integradora: base de datos `clinica_medica`
9. Resumen rápido para repasar


---

## 1. ¿Qué es SQL?

### 1.1 Definición

**SQL** son las siglas de **Structured Query Language** (Lenguaje de Consulta Estructurada).

Es el **lenguaje estándar para interactuar con bases de datos relacionales**: obtener, calcular, actualizar o eliminar datos de forma precisa y eficiente.

En simple: SQL es el idioma con el que le pedimos cosas a una base de datos (consultar, insertar, modificar, borrar).

### 1.2 Principales motores que lo usan

| Motor | Característica destacada |
|---|---|
| **MySQL** | El más popular del mundo |
| **SQLite** | Ligero, ideal para apps móviles |
| **MariaDB** | Fork de MySQL, 100% open source |

> Aunque cada motor tiene sus particularidades, **todos comparten la misma base de SQL**.

### 1.3 SQL: un lenguaje cercano al inglés

Una de las grandes ventajas de SQL es su **legibilidad**. Sus sentencias están inspiradas en el idioma inglés, lo que facilita enormemente su aprendizaje. Aunque ha evolucionado a lo largo de los años, su estructura fundamental sigue siendo la misma desde sus inicios.

Palabras clave que aparecen en el material:

| Sentencia | Significado |
|---|---|
| **CREATE** | Crear tablas o bases de datos |
| **SELECT** | Consultar y recuperar datos |
| **INSERT** | Insertar nuevos registros |
| **DELETE** | Eliminar datos existentes |
| **GROUP BY** | Agrupar resultados |

> La sintaxis de SQL se lee casi como una frase en inglés:
>
> `SELECT nombre FROM personas WHERE id = 1`
>
> → "Selecciona el nombre de personas donde el id sea 1".

---

# 2. Tipos de sentencias SQL: DDL, DML y DCL

SQL organiza sus instrucciones en **tres grandes categorías según su propósito**: definir estructuras, manipular datos o controlar el acceso. Conocer esta clasificación es clave para entender qué hace cada sentencia.

| Familia | Nombre completo | Propósito | Sentencias |
|---|---|---|---|
| **DDL** | Data Definition Language | Define y modifica la **estructura** de la base de datos: tablas, campos, índices... | `CREATE` · `ALTER` · `DROP` · `TRUNCATE` |
| **DML** | Data Manipulation Language | Opera sobre los **datos** almacenados: consultar, insertar, modificar o borrar registros | `SELECT` · `INSERT` · `UPDATE` · `DELETE` |
| **DCL** | Data Control Language | Gestiona los **permisos y accesos** de usuarios y roles dentro del sistema | `GRANT` · `REVOKE` |

## 2.1 DDL · Data Definition Language — definiendo estructuras

Las sentencias DDL permiten crear y modificar la **arquitectura** de la base de datos. **No trabajan con los datos en sí, sino con los contenedores que los almacenan.**

- **CREATE** — Crea nuevos objetos en la base de datos: tablas, campos, bases de datos completas, índices, vistas...
- **ALTER** — Modifica la estructura de objetos ya existentes, por ejemplo añadir o eliminar una columna de una tabla.
- **DROP** — Elimina por completo objetos de la base de datos (tablas, campos, bases de datos). **¡Cuidado: es irreversible!**
- **TRUNCATE** — Vacía todos los datos de una tabla, pero **conserva su estructura**. Más rápido que `DELETE` para borrar todo.

## 2.2 DML · Data Manipulation Language — trabajando con los datos

Las sentencias DML son las que más utilizarás en el día a día. Permiten **leer, agregar, modificar y eliminar registros** dentro de las tablas ya creadas.

- **SELECT** — Recupera y consulta datos de una o varias tablas.
- **INSERT** — Inserta nuevas filas de datos en una tabla existente.
- **UPDATE** — Actualiza los valores de registros ya existentes en la tabla.
- **DELETE** — Elimina filas concretas de una tabla, pudiendo filtrar cuáles.

### 2.3 DCL · Data Control Language — controlando el acceso

- **GRANT** — Concede permisos a uno o varios usuarios (o roles) para realizar operaciones determinadas sobre objetos de la base de datos, como consultar una tabla, insertar datos o ejecutar procedimientos.

```
GRANT SELECT ON personas TO usuario1;
```

- **REVOKE** — Revoca o elimina permisos que previamente se habían concedido con `GRANT`. Si un usuario ya no necesita cierto acceso, se utiliza `REVOKE` para retirárselo de forma segura.

```
REVOKE SELECT ON personas FROM usuario1;
```

> Gestionar bien los permisos es **fundamental para la seguridad** de la base de datos.

---

# 3. Cláusulas, operadores y condiciones

## 3.1 Cláusulas

| Cláusula | Descripción |
|---|---|
| **FROM** | Utilizada para especificar la tabla de la cual se van a seleccionar los registros. |
| **WHERE** | Utilizada para especificar las condiciones que deben reunir los registros que se van a selección. |
| **ORDER BY** | Utilizada para ordenar los registros seleccionados de acuerdo con un orden específico. |

## 3.2 Operadores de comparación

| Operador | Significado |
|---|---|
| `=` | Igual a |
| `>` | Mayor que |
| `<` | Menor que |
| `>=` | Mayor o igual que |
| `<=` | Menor o igual que |
| `<>` | No es igual a |
| `!=` | No es igual a (No es del estándar ISO) |

## 3.3 Operadores lógicos

| Operador | Significado |
|---|---|
| **AND** | `TRUE` si **ambas** expresiones booleanas son `TRUE`. |
| **OR** | `TRUE` si **cualquiera** de las dos expresiones booleanas es `TRUE`. |

- "**AND es restrictivo**: exige que **todas** las condiciones se cumplan."
- "**OR es inclusivo**: con que se cumpla **una**, alcanza."

**Tabla de verdad del operador AND**

| A | B | Res |
|---|---|---|
| false | false | false |
| true | false | false |
| false | true | false |

| A | B | Res |
|---|---|---|
| true | true | **true** |

**Tabla de verdad del operador OR**

| A | B | Res |
|---|---|---|
| false | false | False |
| true | false | true |
| false | true | true |
| true | true | true |

**Ejemplos del material:**

- **AND:** "El estudiante aprobó matemática **y** aprobó programación → pasa de año."
- **OR:** "El estudiante aprobó matemática **o** programación → puede acceder al curso extra."

## 3.4 Condiciones con palabras clave

| Condición | Significado |
|---|---|
| **BETWEEN** | Especifica en un rango, intervalo. |
| **IN** | Especifica conjunto de valores exactos con los que se va a trabajar. |
| **LIKE** | Determina si una cadena de caracteres específica coincide con un patrón especificado. Un patrón puede contener caracteres normales y caracteres comodín. |
| **IS NULL** | Se utiliza cuando se necesita trabajar con valores `NULL`. En una Base de datos se usa el valor `NULL` cuando el valor de una columna se desconoce o falta. |
| **IS NOT NULL** | Se utiliza cuando se necesita trabajar con valores que no son `NULL`. En una Base de datos se usa el valor `NOT NULL` cuando el valor de una columna no puede faltar. |
| **NOT** | Permite negar las condiciones. |

## 3.5 Sintaxis SQL en la práctica

Veamos cómo se escriben las consultas más habituales. La sintaxis es siempre coherente: **primero indicamos qué queremos hacer, luego sobre qué tabla y opcionalmente con qué condición.**

**SELECT — Consultar datos**

```
SELECT IDPersona, Nombre
FROM personas
WHERE idPersona = 1;
```

> Recupera el ID y el nombre de la persona cuyo `idPersona` es 1.

**INSERT — Insertar un registro**

```
INSERT INTO personas (nombre, apellido)
VALUES ('Elisabeth', 'Pose');
```

> Añade una nueva fila con el nombre Elisabeth Pose en la tabla `personas`.

**DELETE — Eliminar un registro**

```
DELETE FROM personas
WHERE idPersona = 1;
```

> Borra el registro cuyo `idPersona` sea 1. **¡Sin `WHERE` se borrarían todos los registros!**

### 3.6 Ejemplos de borrado

**Sin cláusula `WHERE`: elimina todas las filas de la tabla.**

```
DELETE FROM Alumnos;
```

**Con cláusula `WHERE`: elimina registros aplicando condición.**

```
DELETE FROM Materia WHERE codMat >= 15;
```

**Borra registros aplicando un subquery.**

```
DELETE FROM Alumnos
WHERE idloc IN (SELECT idloc FROM Localidad WHERE DescLoc LIKE 'C%');
```

**Diferencias entre `TRUNCATE` y `DELETE`**

```
TRUNCATE TABLE Cursos;
```

| TRUNCATE | DELETE |
|---|---|
| Resetea el campo identity de la tabla | No resetea el campo identity de la tabla |
| Bloquea la tabla completa | Bloquea solo la fila a borrar |
| Es un comando **DDL** | Es un comando **DML** |
| No se puede usar `WHERE` | Se puede usar `WHERE` para filtrar filas |

---

## 4. Tipos de datos en MySQL

| Tipo de dato | Definición sencilla | Ejemplo práctico |
|---|---|---|
| **CHAR(n)** | Texto de longitud **fija**. Siempre ocupa n caracteres. Si sobra, rellena con espacios. | `CREATE TABLE t (codigo CHAR(5));`<br>`INSERT INTO t VALUES ('AB');` -- se guarda `'AB   '` |
| **VARCHAR(n)** | Texto de longitud **variable**, hasta n caracteres. Solo usa el espacio necesario. | `CREATE TABLE t (nombre VARCHAR(10));`<br>`INSERT INTO t VALUES ('Ana');` -- `'Ana'`<br>`INSERT INTO t VALUES ('Juan Perez');` -- `'Juan Perez'` |
| **INT** | Número **entero**. El `(n)` como en `INT(5)` **no limita dígitos**, solo servía para `ZEROFILL` (hoy ya no se usa). Rango: `-2,147,483,648` a `2,147,483,647`. | `CREATE TABLE t (edad INT);`<br>`INSERT INTO t VALUES (25);`<br>`INSERT INTO t VALUES (1234567890);` |
| **DECIMAL(p,s)** | Número **exacto** con decimales. `p` = total de dígitos, `s` = cuántos van después de la coma. Muy usado en dinero. | `CREATE TABLE t (precio DECIMAL(10,2));`<br>`INSERT INTO t VALUES (12345678.90);` -- válido (8 enteros + 2 decimales)<br>`INSERT INTO t VALUES (123456789.90);` -- error (11 dígitos en total) |

| Tipo de dato | Definición sencilla | Ejemplo práctico |
|---|---|---|
| **FLOAT** | Número decimal **aproximado** (puede perder precisión). Útil en cálculos científicos y mediciones. | `CREATE TABLE t (altura FLOAT);`<br>`INSERT INTO t VALUES (1.75);`<br>`INSERT INTO t VALUES (65.123456);` |
| **DATE** | Guarda una fecha en formato `YYYY-MM-DD`. | `CREATE TABLE t (fecha DATE);`<br>`INSERT INTO t VALUES ('2025-09-15');`<br>`INSERT INTO t VALUES ('2000-01-01');` |
| **BOOLEAN** | Valor lógico `TRUE`/`FALSE`. Internamente MySQL lo guarda como `TINYINT(1)` (1 o 0). | `CREATE TABLE t (activo BOOLEAN);`<br>`INSERT INTO t VALUES (TRUE);` -- guarda 1<br>`INSERT INTO t VALUES (FALSE);` -- guarda 0 |
| **ENUM** | Lista **cerrada** de valores posibles. Si insertás algo fuera de la lista → error. Internamente guarda un número índice. | `CREATE TABLE t (estado ENUM('pendiente','proceso','finalizado'));`<br>`INSERT INTO t VALUES ('pendiente');`<br>`INSERT INTO t VALUES ('finalizado');`<br>`INSERT INTO t VALUES ('otro');` -- error, no está en la lista |
| **TEXT** | Texto **largo** de longitud variable. No requiere especificar un tamaño máximo, admite hasta **65.535 caracteres**. | `CREATE TABLE t (descripcion TEXT);`<br>`INSERT INTO t VALUES ('Este es un texto corto');`<br>`INSERT INTO t VALUES ('Este es un texto mucho más largo, que puede ocupar varias oraciones o párrafos completos, ideal para descripciones extensas.');` |

> Se usa para representar datos que solo pueden tomar ciertos valores predefinidos. — (definición del tipo ENUM)

### Importante — ideas para recordar

- **CHAR = fijo** → ideal para códigos que siempre tienen la misma longitud.
- **VARCHAR = variable** → ideal para nombres, descripciones.
- **INT(n)** → el `(n)` ya no sirve como límite, solo era formato.
- **DECIMAL(p,s)** → precisión total (`p`), con `s` decimales.
- **FLOAT** → rápido pero puede perder precisión.
- **BOOLEAN** → en realidad es `0` o `1`.
- **ENUM** → práctico para valores de un conjunto fijo, pero difícil de modificar si después querés agregar opciones.
- **TEXT** → pensado para texto largo, no podés definir longitud máxima ni usarlo como clave.

---

# 5. Constraints en SQL

## 5.1 Definición

En SQL, los **constraints (restricciones)** son reglas que se aplican a las tablas o columnas para que los datos se guarden correctamente y no haya errores.

Sirven para:

- Evitar datos repetidos
- Evitar datos incorrectos
- Mantener relaciones entre tablas

> "Son reglas que obligan a que los datos estén bien cargados"

## 5.2 Tipos de constraints

### PRIMARY KEY

- Identifica de forma única cada fila.
- No se puede repetir.
- No puede ser `NULL`.

```
id_cliente INT PRIMARY KEY
```

> "Es el DNI de la tabla"

### FOREIGN KEY

- Relaciona una tabla con otra.
- Solo permite valores que existan en la tabla padre.

```
FOREIGN KEY (id_cliente) REFERENCES clientes(id_cliente)
```

> "No podés poner un cliente que no exista"

### UNIQUE

- Evita que se repitan valores.
- Similar a la PK, pero puede haber `NULL`.

```
dni CHAR(8) UNIQUE
```

> "No se puede repetir, pero no es la clave principal"

### NOT NULL

- No permite valores vacíos.

```
nombre VARCHAR(50) NOT NULL
```

> "Este dato es obligatorio"

### CHECK

- Define una condición que debe cumplirse.

```
precio DECIMAL(10,2) CHECK (precio > 0)
```

> "El dato tiene que cumplir una regla"

### DEFAULT

- Define un valor por defecto.

```
activo BOOLEAN DEFAULT TRUE
```

> "Si no ponés nada, se completa solo"

## 5.3 Foreign Key con acciones

Se puede definir qué pasa cuando se modifica o elimina un dato en la **tabla padre**:

```
CONSTRAINT nombre_fk
FOREIGN KEY (columna_hija)
REFERENCES tabla_padre (columna_padre)
ON DELETE <acción>
ON UPDATE <acción>;
```

**Acciones posibles:**

  - Ejemplo: si borrás un estudiante → se borran sus inscripciones.
  - "Se borra todo en cadena"
- **CASCADE** — Si borrás o actualizás en la tabla padre → se hace lo mismo en la hija.
  - Solo funciona si permite `NULL`.
  - "Se borra el dato, pero no el registro"
- **SET NULL** — Si borrás en la tabla padre → en la hija queda `NULL`.
  - "No te deja borrar porque está siendo usado"
- **RESTRICT** — No te deja borrar si hay datos relacionados.

### 5.4 Ejemplo completo: tabla telefonos

```
CREATE TABLE telefonos (
    id_telefono INT PRIMARY KEY AUTO_INCREMENT,
    id_estudiante INT,
    telefono VARCHAR(20),
    FOREIGN KEY (id_estudiante) REFERENCES estudiantes(id_estudiante)
);
```

> En este ejemplo se combinan varios constraints a la vez: `PRIMARY KEY` (con `AUTO_INCREMENT`, que genera automáticamente el número siguiente), un campo simple y una `FOREIGN KEY` que relaciona cada teléfono con un estudiante existente.

### 5.5 Idea de cierre

> Los constraints son reglas que evitan errores: controlan que los datos no se repitan, no estén vacíos, sean válidos y que las tablas estén bien relacionadas.

---

## 6. JOIN: combinar tablas

### 6.1 ¿Qué es un JOIN?

**Interacción entre tablas.**

Los **JOINS en SQL sirven para combinar filas de dos o más tablas basándose en un campo común entre ellas**. Un JOIN devuelve datos de diferentes tablas simultáneamente.

Ejemplo visual del material: la tabla `estudiantes` y la tabla `inscripciones` comparten el campo común `id_estudiante`, y por ese campo se pueden relacionar.

> Existen diferentes tipos de JOINS.

### 6.2 Cláusula INNER JOIN

> **Importante: SOLO SE MOSTRARÁ LA INTERSECCIÓN EN LOS RESULTADOS.**

El **INNER JOIN** devuelve **sólo las filas que tienen coincidencias en ambas tablas**.

**Ejemplo del material:**

`EMPLEADOS`

| id_empleado | nombre |
|---|---|
| 1 | Ana |
| 2 | Luis |
| 3 | Marta |

`DEPARTAMENTOS`

| id_departamento | id_empleado | departamento |
|---|---|---|
| 1 | 1 | Ventas |
| 2 | 3 | Marketing |
| 3 | 4 | Finanzas |

**Resultado INNER JOIN**

| nombre | departamento |
|---|---|
| Ana | Ventas |
| Marta | Marketing |

> Luis no aparece porque no tiene coincidencia en `DEPARTAMENTOS`, y Finanzas no aparece porque su `id_empleado` (4) no existe en `EMPLEADOS`. Solo queda la intersección.

## 6.3 Cláusula LEFT JOIN

El **LEFT JOIN** devuelve **todas las filas de la tabla izquierda** (en este caso 'Empleados'), y las coincidencias de la tabla derecha ('Departamentos'). **Si no hay coincidencia, devuelve `NULL`.**

```
SELECT empleados.nombre, departamentos.departamento
FROM empleados
LEFT JOIN departamentos
ON empleados.id_empleado = departamentos.id_empleado;
```

**Resultado LEFT JOIN** (con las mismas tablas del ejemplo anterior)

| nombre | departamento |
|---|---|
| Ana | Ventas |
| Luis | NULL |
| Marta | Marketing |

> Luis se conserva aunque no tenga departamento: su departamento queda `NULL`.

## 6.4 Cláusula RIGHT JOIN

El **RIGHT JOIN** devuelve **todas las filas de la tabla derecha** (en este caso 'Departamentos'), y las coincidencias de la tabla izquierda ('Empleados'). **Si no hay coincidencia, devuelve `NULL`.**

**Resultado RIGHT JOIN** (con las mismas tablas del ejemplo anterior)

| nombre | departamento |
|---|---|
| Ana | Ventas |
| Marta | Marketing |
| NULL | Finanzas |

> Finanzas se conserva aunque no tenga empleado: el nombre queda `NULL`.

### 6.5 Cláusula FULL JOIN

El **FULL JOIN** devuelve **todas las filas de ambas tablas**, emparejando cuando hay coincidencias, y completando con `NULL` cuando no las hay.

### 6.6 Tipos de JOINS en SQL — resumen comparativo

| Tipo | Qué devuelve |
|---|---|
| **(INNER) JOIN** | Devuelve los registros que tienen valores coincidentes en ambas tablas. (Solo la intersección.) |
| **LEFT JOIN** | Devuelve todos los registros de la tabla izquierda y los registros coincidentes de la tabla derecha. |
| **RIGHT JOIN** | Devuelve todos los registros de la tabla derecha y los registros coincidentes de la tabla izquierda. |
| **FULL JOIN** | Devuelve todas las filas de ambas tablas, emparejando cuando hay coincidencias y completando con `NULL` cuando no las hay. |

> Truco para recordarlos con los diagramas de Venn del material: **INNER** = solo el centro (intersección) · **LEFT** = todo el círculo izquierdo + centro · **RIGHT** = centro + todo el círculo derecho · **FULL** = los dos círculos completos.

---

## 7. Agregaciones y GROUP BY

> **Tema fuera del material de clase**, pero es el complemento natural de las funciones de agregación (`COUNT · SUM · AVG · MAX · MIN`) y de la práctica de la sección 8: casi todos los enunciados de `clinica_medica` terminan agrupando. Todo lo de abajo está verificado ejecutando las consultas sobre esa base.

### 7.1 La idea en una frase

`GROUP BY` **parte las filas en "cajones" y devuelve una sola fila por cajón.** Sin `GROUP BY`, la tabla entera es un único cajón.

```
SELECT COUNT(*) AS pacientes, ROUND(AVG(edad),1) AS edad_prom FROM pacientes;
-- 15 · 47.4                                              → 1 fila

SELECT ciudad, COUNT(*) AS pacientes, ROUND(AVG(edad),1) AS edad_prom
FROM pacientes
GROUP BY ciudad
ORDER BY pacientes DESC, ciudad;
-- Rosario 3 (58.7) · Buenos Aires 2 (37) · Córdoba 2 (26.5) · Mendoza 2 (48)
-- Salta 2 (50) · La Plata 1 (54) · Neuquén 1 (67) · San Juan 1 (58) · Tucumán 1 (33)
--                                                          → 9 filas
```

El `COUNT(*)` deja de contar "todos" y **cuenta dentro de cada cajón**; lo mismo pasa con `AVG`, `SUM`, `MAX` y `MIN`.

### 7.2 El vocabulario

| Término | Qué es |
|---|---|
| **clave de agrupamiento** | lo que va después de `GROUP BY`; define los cajones (`ciudad`). |
| **función de agregación** | reduce N filas a un valor: `COUNT · SUM · AVG · MAX · MIN · GROUP_CONCAT`. |
| **clave compuesta** | agrupar por 2 o más columnas: el cajón es la combinación (`GROUP BY ciudad, rango`). |
| **agrupar por expresión** | el cajón es un cálculo, no una columna: `GROUP BY YEAR(fecha)` o un `CASE`. |

| Término | Qué es |
|---|---|
| **filtro de filas** | `WHERE`: se aplica **antes** de armar los cajones. |
| **filtro de grupos** | `HAVING`: se aplica **después**, sobre cada grupo ya formado. |
| `ONLY_FULL_GROUP_BY` | modo de MySQL (activo desde 5.7) que exige la regla de oro; si se incumple da **error 1055**. |
| **dependencia funcional** | la excepción que habilita MySQL: si agrupás por la **PK**, podés mostrar las otras columnas de esa tabla. |
| **agregado ponderado** | el efecto de joinear antes de agregar: las filas duplicadas "pesan más" en el promedio. |

## 7.3 Las funciones de agregación y los NULL

| Función | Qué hace | Con NULL | Medido en clinica_medica |
|---|---|---|---|
| `COUNT(*)` | cuenta **filas** | las cuenta | `medicos LEFT JOIN turnos` → **37** |
| `COUNT(col)` | cuenta valores **no nulos** | los ignora | `COUNT(t.id_turno)` → **30** (faltan 7 = los médicos sin turno) |
| `COUNT(DISTINCT col)` | valores distintos | ignora `NULL` | `COUNT(DISTINCT ciudad)` → **9** |
| `SUM(col)` | suma | ignora `NULL` | `SUM(m.salario)` por especialidad → 1.130.000 (Neurología) |
| `AVG(col)` | promedio de los no nulos | divide por la cantidad **de no nulos**, no por `COUNT(*)` | `AVG(salario)` → 483.666,67 |
| `MAX` / `MIN` | extremo; andan con número, `DATE` y texto | ignoran `NULL` | `MAX(fecha_turno)` → 2025-10-30 |
| `GROUP_CONCAT(col)` (MySQL) | junta los valores en un string | ignora `NULL` | Rosario → `'Juan,Laura,Pedro'` |

> `ROUND(x, 2)` **no** es una agregación: solo redondea el resultado. Úsala encima de `AVG` para que el promedio se lea.

## 7.4 La regla de oro

**Toda columna del `SELECT` que no esté dentro de una función de agregación tiene que estar en el `GROUP BY`.**

No es un capricho del profe: cada cajón representa a muchas filas, y una columna suelta **no tiene un valor único**.

```
SELECT ciudad, nombre, COUNT(*) AS n FROM pacientes GROUP BY ciudad;   -- MAL
```

SQLite la deja correr y devuelve `Buenos Aires | María | 2`... ¿María o Jorge? Son dos pacientes: eligió uno al azar, **el resultado miente**. MySQL con `ONLY_FULL_GROUP_BY` la corta en seco:

```
ERROR 1055 (42000): Expression #2 of SELECT list is not in GROUP BY clause
```

Tres formas de corregirla:

```
-- a) sacarla
SELECT ciudad, COUNT(*) AS n FROM pacientes GROUP BY ciudad;

-- b) agrupar también por ella (el cajón pasa a ser ciudad + nombre)
SELECT ciudad, nombre, COUNT(*) AS n FROM pacientes GROUP BY ciudad, nombre;

-- c) agregarla (forzar un valor por grupo)
SELECT ciudad, GROUP_CONCAT(nombre) AS nombres, COUNT(*) AS n
FROM pacientes GROUP BY ciudad;      -- Rosario → 'Juan,Laura,Pedro' · 3
```

Y la excepción que conviene conocer: **si el** `GROUP BY` **usa la clave primaria**, MySQL deja seleccionar cualquier otra columna de esa tabla (es funcionalmente dependiente). Por eso en el material se agrupa `GROUP BY m.id_medico` y se muestra `m.nombre`.

## 7.5 El orden de ejecución: el que resuelve todas las dudas

Se **escribe** `SELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY → LIMIT`, pero la base **ejecuta**:

```
FROM  →  WHERE  →  GROUP BY  →  HAVING  →  SELECT  →  ORDER BY  →  LIMIT
 1         2          3            4           5           6          7
                                  (los alias nacen acá)
```

De ese orden se desprende todo lo que suele confundir:

- En el `WHERE` todavía **no existe** `COUNT(*)`: `WHERE COUNT(*) > 1` da `ERROR 1111 (Invalid use of group function)`. Las agregadas se filtran en `HAVING` (paso 4).
- Los **alias del** `SELECT` **nacen en el paso 5**: no se pueden usar en `WHERE` (`ERROR 1054: Unknown column 'n' in 'where clause'`), pero **sí** en `HAVING` y en `ORDER BY` (MySQL lo permite).
- `ORDER BY` se aplica a los grupos ya formados: por eso ordena por el agregado (`ORDER BY n DESC`).

## 7.6 WHERE y HAVING, con el mismo objetivo y resultados distintos

```
-- filtra FILAS antes de agrupar: cuenta solo los mayores de 40
SELECT ciudad, COUNT(*) AS n
FROM pacientes WHERE edad > 40 GROUP BY ciudad;
-- Rosario 3 · Mendoza 2 · La Plata 1 · Neuquén 1 · Salta 1 · San Juan 1  → 6 filas

-- filtra GRUPOS después: cuenta a todos y tira los cajones chicos
SELECT ciudad, COUNT(*) AS n
FROM pacientes GROUP BY ciudad HAVING COUNT(*) > 1;
-- Rosario 3 · Buenos Aires 2 · Córdoba 2 · Mendoza 2 · Salta 2            → 5 filas

-- los dos: primero filas, después grupos
SELECT ciudad, COUNT(*) AS n
FROM pacientes WHERE edad > 40
GROUP BY ciudad HAVING COUNT(*) > 1;
-- Rosario 3 · Mendoza 2                                                  → 2 filas
```

## 7.7 La plantilla

```
SELECT   columna_grupo,                        -- solo lo que está en GROUP BY…
         AGREGACION(otra_columna) AS alias      -- …o dentro de una agregada
FROM     tabla
WHERE    filtro_sobre_filas                     -- opcional, sin agregadas
GROUP BY columna_grupo                          -- arma los cajones
HAVING   filtro_sobre_grupos                    -- opcional, acá sí agregadas
ORDER BY alias DESC, columna_grupo              -- desempatar siempre
LIMIT    n;                                     -- opcional
```

## 7.8 Los patrones que hay que tener domados

**Varias agregadas en la misma consulta**

```
SELECT ciudad, MAX(edad) AS mayor, MIN(edad) AS menor
FROM pacientes
GROUP BY ciudad
ORDER BY (MAX(edad) - MIN(edad)) DESC;
-- Salta 70/30 (brecha 40) · Rosario 73/41 · Mendoza 51/45 · Buenos Aires 39/35 · …
```

`COUNT(*)` **o "cuáles":** `COUNT(DISTINCT x)`

```
SELECT e.nombre_especialidad,
       COUNT(t.id_turno) AS turnos,
       COUNT(DISTINCT t.id_paciente) AS pacientes_distintos
FROM turnos t
INNER JOIN medicos m ON t.id_medico = m.id_medico
INNER JOIN especialidades e ON m.id_especialidad = e.id_especialidad
GROUP BY e.nombre_especialidad;
-- Cardiología 4/4 · Clínica General 4/4 · Dermatología 4/4 · Neurología 4/4 ·
    Pediatría 4/4
-- Traumatología 4/4 · Ginecología 3/3 · Oftalmología 3/3      (la Σ de los grupos da
    30 = el total, así que el JOIN no duplicó)
```

Sumar los grupos y comparar contra el total real es la mejor manera de comprobar que un `JOIN` no duplicó filas.

**Rangos que no existen como columna:** `CASE`

```
SELECT CASE WHEN edad < 40 THEN '18 a 39'
            WHEN edad < 60 THEN '40 a 59'
            ELSE '60 y más' END AS rango,
       COUNT(*) AS pacientes, ROUND(AVG(edad),1) AS edad_prom
FROM pacientes
GROUP BY rango
ORDER BY rango;
-- 18 a 39 → 6 (31.7) · 40 a 59 → 5 (49.8) · 60 y más → 4 (68)
```

**Cortar una fecha para agrupar**

```
SELECT SUBSTR(fecha_turno, 1, 7) AS mes, COUNT(*) AS turnos
FROM turnos GROUP BY mes;
-- 2025-10 → 30  (en MySQL: DATE_FORMAT(fecha_turno,'%Y-%m') o YEAR() + MONTH())
```

**"Los que no tienen nada":** `LEFT JOIN` **+** `HAVING COUNT(...) = 0`

```
SELECT m.nombre, m.apellido, COUNT(t.id_turno) AS turnos
FROM medicos m LEFT JOIN turnos t ON t.id_medico = m.id_medico
GROUP BY m.id_medico
HAVING COUNT(t.id_turno) = 0;
-- 7 filas: Herrera · López · Martínez · Molina · Ortega · Rivas · Silva
```

Con `INNER JOIN` da **0 filas**: el `INNER` descarta justamente las filas sin turno, que son las que estabas buscando. Y ojo con `COUNT(*)` acá: sobre el `LEFT JOIN` devuelve **1** para esos médicos (la fila con `NULL`), mientras que `COUNT(t.id_turno)` devuelve **0**.

**Porcentaje sobre el total, con subconsulta en el denominador**

```
SELECT e.nombre_especialidad, COUNT(*) AS turnos,
       ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM turnos), 1) AS porcentaje
FROM turnos t
INNER JOIN medicos m ON t.id_medico = m.id_medico
INNER JOIN especialidades e ON m.id_especialidad = e.id_especialidad
GROUP BY e.nombre_especialidad;
-- 13.3 % (seis especialidades) · 10 % (dos)  → el total da 100 %
```

El `* 100.0` (con punto decimal) evita la división entera; y el total se saca con un subquery **aparte**, no con `COUNT(*)` a secas.

## 7.9 La trampa del JOIN que sesga el promedio

```
SELECT ROUND(AVG(salario),2) FROM medicos;                          -- 483666.67
SELECT ROUND(AVG(salario),2) FROM medicos
WHERE id_medico IN (SELECT id_medico FROM turnos);                  -- 481875
SELECT ROUND(AVG(m.salario),2)
FROM medicos m INNER JOIN turnos t ON t.id_medico = m.id_medico;  -- 480166.67
```

Hay **dos efectos mezclados**:

1. el `JOIN` **excluye** a los 7 médicos sin turnos → el promedio pasa a ser de 8, no de 15;
2. y **repite** el sueldo de cada médico una vez por turno → queda **ponderado**:

```
peso 4: Luis 420000 · Valeria 430000 · Andrés 450000 · Pedro 470000 · Carla 510000 ·
    Romina 560000
peso 3: Lucas 495000 · Marina 520000
```

Ninguno está "mal": son preguntas distintas. Antes de agregar sobre un `JOIN` de 1:N, definí qué querés medir y, si hace falta, **agrupá primero en un subquery y joineá después**.

## 7.10 Errores típicos y su antídoto

| Síntoma | Causa | Arreglo |
|---|---|---|
| `ERROR 1055 … not in GROUP BY clause` | columna suelta en el `SELECT` | sacarla, agruparla o meterla en una agregada |
| `ERROR 1111 Invalid use of group function` | una agregada en el `WHERE` | moverla al `HAVING` |
| `ERROR 1054 Unknown column 'n'` | usaste un alias del `SELECT` en el `WHERE` | usarlo en `HAVING`/`ORDER BY` o repetir la expresión |
| el `AVG` o el `SUM` dan más grande/chico de lo esperado | el `JOIN` 1:N duplicó filas | verificar con `COUNT(*)` contra el total, o agregar antes de joinear |
| `COUNT(*)` dice 1 pero "no hay datos" | `LEFT JOIN` sin coincidencia (fila `NULL`) | contar con `COUNT(clave_hija)` |
| un grupo desaparece de la lista | `INNER JOIN` filtró los que no tenían nada | `LEFT JOIN` + `HAVING COUNT(...) = 0` |
| el orden de los empates cambia cada vez | `ORDER BY` sin desempate | `ORDER BY n DESC, ciudad` |
| SQLite deja hacer algo que MySQL rechaza | SQLite no valida `ONLY_FULL_GROUP_BY` | no confiar en el resultado "arbitrario" |

## 7.11 Checklist para el parcial

1. Definí **qué fila** querés que devuelva la consulta: esa es la clave del `GROUP BY`.
2. Lo que no esté en esa clave, solo dentro de una función de agregación.
3. Filtro de filas → `WHERE`. Filtro de grupos → `HAVING`.

4. ¿Joinear antes o después de agregar? Si el `JOIN` duplica filas, agregá primero.
5. `ORDER BY alias DESC, clave_de_grupo` para que el resultado sea estable.
6. Sumá los grupos y compará contra el total real: si no cierra, hay filas duplicadas por el `JOIN`.
7. `ROUND(x, 2)` para promedios y `* 100.0` para porcentajes.

> Para practicar esto con resultados verificables: `practica/group-by-ejercicios.md` (15 ejercicios) y `practica/run.py`, que además te avisa si tu consulta reventaría con `ONLY_FULL_GROUP_BY`.

---

# 8. Práctica integradora: base de datos clinica_medica

> Fuente: «**DATABASE clinica_medica — práctica de todas las consultas**».
>
> Es la práctica más completa del material porque recorre **todos los temas** del resumen en una sola base de datos: DDL (`CREATE TABLE`), constraints (`PRIMARY KEY`, `AUTO_INCREMENT`, `FOREIGN KEY`), tipos de datos (`INT`, `VARCHAR`, `DECIMAL`, `DATE`), DML (`SELECT`, `INSERT`, `UPDATE`, `DELETE`), `ALTER TABLE`, operadores y palabras clave, funciones de agregación, `GROUP BY` / `HAVING`, los tres `JOIN` y `CREATE VIEW`.
>
> Los 56 enunciados están resueltos abajo. **Todas las consultas fueron ejecutadas sobre los datos reales del PDF**, así que en 8.12 figura la cantidad de filas (o el valor) que tiene que dar cada una para que puedas autocorregirte.

## 8.1 Estructura de la base (DDL + constraints)

Cuatro tablas. `turnos` es la tabla "puente": relaciona pacientes con médicos, y cada médico pertenece a una especialidad.

| Tabla | Campos | Constraints |
|---|---|---|
| `especialidades` | `id_especialidad`, `nombre_especialidad` | PK · `AUTO_INCREMENT` · `NOT NULL` |
| `pacientes` | `id_paciente`, `nombre`, `apellido`, `edad`, `ciudad` | PK · `AUTO_INCREMENT` |
| `medicos` | `id_medico`, `nombre`, `apellido`, `salario`, `id_especialidad` | PK · `AUTO_INCREMENT` · FK → `especialidades` |
| `turnos` | `id_turno`, `id_paciente`, `id_medico`, `fecha_turno`, `motivo` | PK · `AUTO_INCREMENT` · 2 FK → `pacientes` y `medicos` |

```
CREATE DATABASE clinica_medica;
USE clinica_medica;

CREATE TABLE especialidades (
    id_especialidad INT AUTO_INCREMENT PRIMARY KEY,
    nombre_especialidad VARCHAR(50) NOT NULL
);

CREATE TABLE pacientes (
    id_paciente INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100),
    apellido VARCHAR(100),
    edad INT,
    ciudad VARCHAR(100)
);

CREATE TABLE medicos (
    id_medico INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100),
    apellido VARCHAR(100),
    salario DECIMAL(10,2),
    id_especialidad INT,
    FOREIGN KEY (id_especialidad) REFERENCES especialidades(id_especialidad)
);

CREATE TABLE turnos (
    id_turno INT AUTO_INCREMENT PRIMARY KEY,
    id_paciente INT,
    id_medico INT,
    fecha_turno DATE,
    motivo VARCHAR(100),
    FOREIGN KEY (id_paciente) REFERENCES pacientes(id_paciente),
    FOREIGN KEY (id_medico) REFERENCES medicos(id_medico)
);
```

**Qué mirar de este DDL (repasa las secciones 2, 4 y 5):**

- `AUTO_INCREMENT`: la clave la genera MySQL, por eso los `INSERT` **no** listan el `id`.
- `salario DECIMAL(10,2)`: dinero → decimal exacto, nunca `FLOAT`.
- `fecha_turno DATE`: formato `YYYY-MM-DD`.
- `VARCHAR(100)` para textos de largo variable; `NOT NULL` solo donde el dato es obligatorio.
- `FOREIGN KEY`: `turnos.id_medico` **solo puede existir si existe en** `medicos` → por eso en 47-49 hay que borrar primero los registros "hijos".

## 8.2 Datos cargados (DML)

```
INSERT INTO especialidades (nombre_especialidad) VALUES
('Clínica General'), ('Cardiología'), ('Traumatología'), ('Dermatología'),
('Pediatría'), ('Neurología'), ('Ginecología'), ('Oftalmología');

INSERT INTO pacientes (nombre, apellido, edad, ciudad) VALUES
('María','González',35,'Buenos Aires'), ('Juan','Pérez',62,'Rosario'),
('Lucía','Ramírez',28,'Córdoba'), ('Diego','Fernández',45,'Mendoza'),
('Ana','Martínez',70,'Salta'), ('Pablo','Ríos',54,'La Plata'),
('Sofía','López',33,'Tucumán'), ('Carlos','Torres',58,'San Juan'),
('Laura','Silva',41,'Rosario'), ('Martín','Suárez',67,'Neuquén'),
('Valeria','Acosta',25,'Córdoba'), ('Jorge','Moreno',39,'Buenos Aires'),
('Natalia','Castro',51,'Mendoza'), ('Pedro','Benítez',73,'Rosario'),
('Camila','Sosa',30,'Salta');

INSERT INTO medicos (nombre, apellido, salario, id_especialidad) VALUES
('Luis','Moreno',420000,1), ('Carla','Ruiz',510000,2), ('Pedro','Luna',470000,3),
('Valeria','Sosa',430000,4), ('Andrés','Pérez',450000,5), ('Romina','Gómez',560000,6),
('Lucas','Benítez',495000,7), ('Marina','Castro',520000,8),
    ('Javier','Molina',410000,1),
('Camila','Ortega',540000,2), ('Tomás','Rivas',460000,3),
    ('Natalia','Herrera',435000,4),
('Sebastián','López',455000,5), ('Gabriela','Martínez',570000,6),
    ('Ricardo','Silva',530000,8);

-- 30 turnos: un paciente + un médico + fecha (octubre 2025) + motivo
INSERT INTO turnos (id_paciente, id_medico, fecha_turno, motivo) VALUES
(1,1,'2025-10-01','Control general'), (2,2,'2025-10-02','Chequeo cardíaco'),
(3,3,'2025-10-03','Dolor de rodilla'), (4,4,'2025-10-04','Problema de piel'),
(5,5,'2025-10-05','Control pediátrico'), (6,6,'2025-10-06','Migrañas'),
(7,7,'2025-10-07','Control ginecológico'), (8,8,'2025-10-08','Control visual'),
(9,1,'2025-10-09','Control general'), (10,2,'2025-10-10','Hipertensión'),
(11,3,'2025-10-11','Fractura'), (12,4,'2025-10-12','Dermatitis'),
(13,5,'2025-10-13','Control clínico'), (14,6,'2025-10-14','Dolor de cabeza'),
(15,7,'2025-10-15','Consulta ginecológica'), (1,8,'2025-10-16','Control visual'),
(2,1,'2025-10-17','Control general'), (3,2,'2025-10-18','Chequeo cardíaco'),
(4,3,'2025-10-19','Traumatismo'), (5,4,'2025-10-20','Alergia'),
(6,5,'2025-10-21','Control clínico'), (7,6,'2025-10-22','Neurología'),
(8,7,'2025-10-23','Control anual'), (9,8,'2025-10-24','Problemas de visión'),
(10,1,'2025-10-25','Control general'), (11,2,'2025-10-26','Chequeo cardíaco'),
(12,3,'2025-10-27','Lesión deportiva'), (13,4,'2025-10-28','Dermatitis'),
(14,5,'2025-10-29','Control médico'), (15,6,'2025-10-30','Estudio neurológico');
```

**Volumen de datos:** 8 especialidades · 15 pacientes · 15 médicos · 30 turnos. Cada turno tiene **un día distinto** (`2025-10-01` al `2025-10-30`) y cada paciente tiene 1 o 2 turnos: los pacientes 1 a 15 tienen un turno en la "vuelta 1" y los pacientes 1 a 15 vuelven en la "vuelta 2" (médicos rotados).

## 8.3 Consultas simples (1 a 12)

1. Mostrar todos los pacientes → `SELECT * FROM pacientes;`
2. Nombre, apellido y edad de todos los pacientes →
   `SELECT nombre, apellido, edad FROM pacientes;`
3. Pacientes mayores de 50 años → `SELECT * FROM pacientes WHERE edad > 50;`
4. Pacientes cuya ciudad sea Rosario → `SELECT * FROM pacientes WHERE ciudad = 'Rosario';`
5. Pacientes menores de 40 años → `SELECT * FROM pacientes WHERE edad < 40;`
6. Médicos ordenados por salario de mayor a menor →
   `SELECT * FROM medicos ORDER BY salario DESC;`
7. Médicos ordenados alfabéticamente por apellido → `SELECT * FROM medicos ORDER BY apellido;`
8. Especialidades ordenadas por nombre →
   `SELECT * FROM especialidades ORDER BY nombre_especialidad;`

9. Salario promedio de los médicos → `SELECT AVG(salario) AS salario_promedio FROM medicos;`
10. Salario máximo y mínimo →
    `SELECT MAX(salario) AS maximo, MIN(salario) AS minimo FROM medicos;`
11. Cantidad total de pacientes → `SELECT COUNT(*) AS total_pacientes FROM pacientes;`
12. Suma total de salarios → `SELECT SUM(salario) AS masa_salarial FROM medicos;`

> **Ojo con el orden lógico de una consulta (sección 3):** se escribe `SELECT … FROM … WHERE … ORDER BY`, pero la base de datos lo ejecuta al revés: primero `FROM`, después `WHERE`, recién después `SELECT` y por último `ORDER BY`. Por eso en el `SELECT` se puede usar el alias y en el `WHERE` no.

## 8.4 Funciones de agregación (13 a 17)

> Acá conviene tener leída la sección 7 (agregaciones) y, para lo que sigue, 7.4 a 7.6.

13. Edad promedio de los pacientes → `SELECT AVG(edad) AS edad_promedio FROM pacientes;`
14. Edad máxima y mínima → `SELECT MAX(edad) AS mayor, MIN(edad) AS menor FROM pacientes;`
15. Cantidad total de turnos → `SELECT COUNT(*) AS total_turnos FROM turnos;`
16. Fecha del primer turno → `SELECT MIN(fecha_turno) AS primer_turno FROM turnos;`
17. Fecha del último turno → `SELECT MAX(fecha_turno) AS ultimo_turno FROM turnos;`

> `MIN()` y `MAX()` no solo sirven con números: con `DATE` (16 y 17) y con texto (alfabéticamente) también funcionan.

## 8.5 Consultas con JOIN (18 a 25)

18. Nombre del paciente y fecha de su turno.

```
SELECT p.nombre AS paciente, t.fecha_turno
FROM turnos t
INNER JOIN pacientes p ON t.id_paciente = p.id_paciente;
```

19. Nombre del médico y su especialidad.

```
SELECT m.nombre AS medico, e.nombre_especialidad
FROM medicos m
INNER JOIN especialidades e ON m.id_especialidad = e.id_especialidad;
```

20. Paciente, médico y motivo del turno (dos JOIN encadenados).

```
SELECT p.nombre AS paciente, m.nombre AS medico, t.motivo
FROM turnos t
INNER JOIN pacientes p ON t.id_paciente = p.id_paciente
INNER JOIN medicos m ON t.id_medico = m.id_medico;
```

21. Paciente y ciudad donde reside → no necesita JOIN, los dos datos están en `pacientes`:
    `SELECT nombre, apellido, ciudad FROM pacientes;`
22. Médico y salario que percibe → mismo caso: `SELECT nombre, apellido, salario FROM medicos;`
23. Paciente, fecha del turno y motivo.

```
SELECT p.nombre AS paciente, t.fecha_turno, t.motivo
FROM turnos t
INNER JOIN pacientes p ON t.id_paciente = p.id_paciente;
```

24. Médico, especialidad y fecha de cada turno.

```
SELECT m.nombre AS medico, e.nombre_especialidad, t.fecha_turno
FROM turnos t
INNER JOIN medicos m ON t.id_medico = m.id_medico
INNER JOIN especialidades e ON m.id_especialidad = e.id_especialidad;
```

25. Paciente, médico y especialidad que lo atendió (cuatro tablas).

```
SELECT p.nombre AS paciente, m.nombre AS medico, e.nombre_especialidad
FROM turnos t
INNER JOIN pacientes p ON t.id_paciente = p.id_paciente
INNER JOIN medicos m ON t.id_medico = m.id_medico
INNER JOIN especialidades e ON m.id_especialidad = e.id_especialidad;
```

> **Truco para no perderse en los JOIN (sección 6):** arrancá siempre por la tabla "puente" (`turnos`) y de ahí unite a cada tabla padre usando su clave foránea. Cada `JOIN` agrega **una** fila más al `FROM`.
>
> Y para practicar `LEFT JOIN`: los 15 médicos tienen turnos, pero si querés ver **los pacientes sin turno** (no hay en estos datos, la consulta igual debe devolver 0 filas) usá
>
> `SELECT p.nombre FROM pacientes p LEFT JOIN turnos t ON t.id_paciente = p.id_paciente WHERE t.id_turno IS NULL;`

## 8.6 Consultas con GROUP BY (26 a 33)

> Toda esta serie se apoya en la sección 7: la regla de oro (7.4), el orden de ejecución (7.5) y `WHERE` vs `HAVING` (7.6).

26. Cantidad de pacientes por ciudad →
    `SELECT ciudad, COUNT(*) AS cantidad FROM pacientes GROUP BY ciudad;`
27. Cantidad de médicos por especialidad.

```
SELECT e.nombre_especialidad, COUNT(m.id_medico) AS medicos
FROM medicos m
INNER JOIN especialidades e ON m.id_especialidad = e.id_especialidad
GROUP BY e.nombre_especialidad;
```

28. Salario promedio por especialidad.

```
SELECT e.nombre_especialidad, AVG(m.salario) AS salario_promedio
FROM medicos m
INNER JOIN especialidades e ON m.id_especialidad = e.id_especialidad
GROUP BY e.nombre_especialidad;
```

29. Cantidad de turnos por médico (con `LEFT JOIN` para que tampoco desaparezca el médico sin turnos).

```
SELECT m.nombre, m.apellido, COUNT(t.id_turno) AS turnos
FROM medicos m
LEFT JOIN turnos t ON t.id_medico = m.id_medico
GROUP BY m.id_medico;
```

30. Cantidad de turnos por fecha →
    `SELECT fecha_turno, COUNT(*) AS turnos_del_dia FROM turnos GROUP BY fecha_turno;`
31. Cantidad de turnos por especialidad.

```
SELECT e.nombre_especialidad, COUNT(t.id_turno) AS turnos
FROM turnos t
INNER JOIN medicos m ON t.id_medico = m.id_medico
INNER JOIN especialidades e ON m.id_especialidad = e.id_especialidad
GROUP BY e.nombre_especialidad;
```

32. "Cantidad de pacientes por rango de ciudad": el enunciado mezcla "rango" con una columna de texto. Dos lecturas posibles:

- Si es **por ciudad**, es idéntica al 26.
- Si la idea es **por rango de edad**, el rango se arma con `CASE`:

```
SELECT CASE
         WHEN edad < 40 THEN '18 a 39'
         WHEN edad < 60 THEN '40 a 59'
         ELSE '60 y más'
       END AS rango_edad,
       COUNT(*) AS cantidad
FROM pacientes
GROUP BY rango_edad;
```

33. Cantidad de turnos realizados por cada paciente.

```
SELECT p.nombre, p.apellido, COUNT(t.id_turno) AS turnos
FROM pacientes p
LEFT JOIN turnos t ON t.id_paciente = p.id_paciente
GROUP BY p.id_paciente;
```

> **Regla de oro del `GROUP BY`:** toda columna que esté en el `SELECT` y que no esté dentro de una función (`COUNT`, `AVG`, `MAX`…) **tiene que estar en el `GROUP BY`**. En MySQL con `ONLY_FULL_GROUP_BY` (viene activo por defecto) una consulta mal armada da error en vez de devolver datos inventados.

## 8.7 Consultas con HAVING (34 a 38)

`HAVING` filtra **después** de agrupar; `WHERE` filtra **antes**. Por eso acá no se puede usar `WHERE COUNT(*) > 1`.

34. Ciudades con más de un paciente.

```
SELECT ciudad, COUNT(*) AS cantidad
FROM pacientes
GROUP BY ciudad
HAVING COUNT(*) > 1;
```

35. Especialidades con más de un médico.

```
SELECT e.nombre_especialidad, COUNT(m.id_medico) AS medicos
FROM especialidades e
INNER JOIN medicos m ON m.id_especialidad = e.id_especialidad
GROUP BY e.nombre_especialidad
HAVING COUNT(m.id_medico) > 1;
```

36. Médicos con más de un turno asignado.

```
SELECT m.nombre, m.apellido, COUNT(t.id_turno) AS turnos
FROM medicos m
INNER JOIN turnos t ON t.id_medico = m.id_medico
GROUP BY m.id_medico
HAVING COUNT(t.id_turno) > 1;
```

37. Especialidades con más de dos turnos.

```
SELECT e.nombre_especialidad, COUNT(t.id_turno) AS turnos
FROM turnos t
INNER JOIN medicos m ON t.id_medico = m.id_medico
INNER JOIN especialidades e ON m.id_especialidad = e.id_especialidad
GROUP BY e.nombre_especialidad
HAVING COUNT(t.id_turno) > 2;
```

38. Fechas con más de una consulta registrada.

```
SELECT fecha_turno, COUNT(*) AS consultas
FROM turnos
GROUP BY fecha_turno
HAVING COUNT(*) > 1;
```

> El 38 **devuelve 0 filas** con estos datos: cada turno tiene un día distinto. No es un error de la consulta, es el dato. Comparalo con el 30: ahí se ven las 30 fechas, todas con `1`.

## 8.8 ALTER TABLE (39 a 41) — estructura, no datos

Estas sentencias son **DDL** (sección 2.1): cambian el contenedor, no los registros.

39. Agregar columna `telefono` a `pacientes`:

```
ALTER TABLE pacientes ADD COLUMN telefono VARCHAR(30);
```

40. Agregar columna `email` a `médicos`:

```
ALTER TABLE medicos ADD COLUMN email VARCHAR(100);
```

41. Renombrar `ciudad` por `localidad` en `pacientes`:

```
ALTER TABLE pacientes RENAME COLUMN ciudad TO localidad;
```

## 8.9 UPDATE (42 a 46) — modificar datos

```
-- 42. Actualizar la edad de un paciente
UPDATE pacientes SET edad = 36 WHERE id_paciente = 1;

-- 43. Modificar la ciudad de un paciente
UPDATE pacientes SET ciudad = 'Córdoba' WHERE id_paciente = 2;

-- 44. Aumentar el salario de TODOS los médicos un 10%
UPDATE medicos SET salario = salario * 1.10;

-- 45. Modificar el motivo de un turno
UPDATE turnos SET motivo = 'Control de seguimiento' WHERE id_turno = 3;

-- 46. Actualizar el salario de un médico específico
UPDATE medicos SET salario = 520000 WHERE id_medico = 1;
```

> Fijate en el 44: **sin `WHERE` el `UPDATE` afecta a toda la tabla** (y acá eso es lo que pide el enunciado). El cálculo usa la columna misma (`salario * 1.10`), por eso alcanza con una sola sentencia.
>
> Si MySQL se niega con "safe update mode", es la protección contra cambios masivos: `SET SQL_SAFE_UPDATES = 0;` o agregá la clave primaria al `WHERE`.

## 8.10 DELETE (47 a 49) — y las foreign keys

```sql
-- 47. Eliminar un turno específico
DELETE FROM turnos WHERE id_turno = 30;

-- 48. Eliminar un paciente que NO tenga turnos asociados
DELETE FROM pacientes
WHERE id_paciente NOT IN (SELECT id_paciente FROM turnos);

-- 49. Eliminar una especialidad que no tenga médicos asignados
DELETE FROM especialidades
WHERE id_especialidad NOT IN (
    SELECT id_especialidad FROM medicos WHERE id_especialidad IS NOT NULL
);
```

> **Por qué el 48 y el 49 llevan el `NOT IN`:** `turnos.id_paciente` es `FOREIGN KEY` de `pacientes`, así que MySQL **no** deja borrar un paciente que tiene turnos (sería `RESTRICT`, sección 5.3). O se borra antes el hijo (el turno) y después el padre, o se declara la FK con `ON DELETE CASCADE`, y ahí el borrado se propaga solo.
>
> Con estos datos: el 48 y el 49 **no borran nada** (los 15 pacientes tienen turnos y las 8 especialidades tienen médicos). Escribilos igual: el profe suele preguntar justo eso.

## 8.11 VISTAS (50 a 56)

Una **vista** es una consulta guardada con nombre: no duplica datos, solo guarda el `SELECT`.

```sql
-- 50. vista_pacientes
CREATE VIEW vista_pacientes AS
SELECT nombre, apellido, edad, ciudad
FROM pacientes;

-- 51. vista_medicos_especialidad
CREATE VIEW vista_medicos_especialidad AS
SELECT m.id_medico, m.nombre, m.apellido, e.nombre_especialidad, m.salario
FROM medicos m
INNER JOIN especialidades e ON m.id_especialidad = e.id_especialidad;

-- 52. vista_turnos_completos (paciente, médico, especialidad, fecha y motivo)
CREATE VIEW vista_turnos_completos AS
SELECT p.nombre AS paciente,
       m.nombre AS medico,
       e.nombre_especialidad,
       t.fecha_turno,
       t.motivo
FROM turnos t
INNER JOIN pacientes p ON t.id_paciente = p.id_paciente
INNER JOIN medicos m ON t.id_medico = m.id_medico
INNER JOIN especialidades e ON m.id_especialidad = e.id_especialidad;
```

Y la gracia de la vista: las consultas "gordas" se leen como una tabla más.

```sql
-- 53. Todos los registros de la vista
SELECT * FROM vista_pacientes;

-- 54. Médicos de Cardiología usando la vista
SELECT * FROM vista_medicos_especialidad
WHERE nombre_especialidad = 'Cardiología';

-- 55. Turnos ordenados por fecha usando la vista
SELECT * FROM vista_turnos_completos
ORDER BY fecha_turno;

-- 56. Turnos por especialidad usando la vista
SELECT nombre_especialidad, COUNT(*) AS turnos
FROM vista_turnos_completos
GROUP BY nombre_especialidad;
```

> Compará el 56 con el 31: dan el mismo resultado, pero el 56 no necesita escribir los cuatro `JOIN` otra vez. Ese es el motivo de existir de las vistas.
>
> Para borrarlas: `DROP VIEW vista_pacientes;` (no borra los datos de las tablas).

## 8.12 Resultado esperado — para autocorregirte

Medido sobre los datos exactos del PDF (`8 · 15 · 15 · 30` registros).

| Ej. | Resultado que tiene que dar |
|---|---|
| 1 | 15 filas |
| 3 | 7 pacientes mayores de 50 |
| 4 | 3 (Juan Pérez, Laura Silva, Pedro Benítez) |
| 5 | 6 menores de 40 |
| 6 | 1.º Gabriela Martínez (570.000) · último Javier Molina (410.000) |
| 9 | 483.666,67 |
| 10 | máx 570.000 · mín 410.000 |
| 11 | 15 |
| 12 | 8.255.000 |
| 13 | 47,4 años |
| 14 | máx 73 · mín 25 |
| 15 | 30 turnos |
| 16 / 17 | `2025-10-01` / `2025-10-30` |
| 18 · 20 · 23 · 24 · 25 | 30 filas (una por turno) |
| 19 · 21 · 22 | 15 filas |
| 26 · 27 · 28 | 9 ciudades · 8 especialidades · 8 promedios |
| 29 | 15 médicos (el máx: Luis Moreno con 4 turnos) |
| 30 | 30 filas, todas con `1` |
| 31 | 8 especialidades · 6 con 4 turnos y 2 con 3 |
| 32 | 18-39 → 6 · 40-59 → 5 · 60 y más → 4 |
| 33 | 15 pacientes |
| 34 | 5 ciudades (Rosario 3 · Buenos Aires, Córdoba, Mendoza y Salta 2) |
| 35 | 7 especialidades (no entra Ginecología, tiene 1 médico) |

| Ej. | Resultado que tiene que dar |
|---|---|
| 36 | 8 médicos |
| 37 | 8 especialidades |
| 38 | **0 filas** (ninguna fecha se repite) |
| 48 · 49 | 0 filas borradas (hay dependencias / no sobra nadie) |
| 53 · 54 · 55 · 56 | 15 filas · 2 médicos (Cardiología) · 30 filas · 8 filas |

### 8.13 Orden sugerido para practicar

1. Crear la base y las tablas (**8.1**) → DDL + constraints.
2. Cargar los datos (**8.2**) → DML `INSERT`.
3. 1-17 sin tocar otra cosa que `SELECT`, `WHERE`, `ORDER BY` y agregaciones.
4. 18-25: un JOIN por vez, y después agregar el siguiente.
5. 26-38: `GROUP BY` primero, `HAVING` después (34 es 26 + filtro).
6. 39-49: acá sí se modifica la base → **conviene hacer una copia o un** `mysqldump` **antes**, porque un `DELETE` no se deshace.
7. 50-56: crear las vistas y reescribir con ellas consultas que ya hiciste.

> Los bloques 39-49 cambian los datos. Si los corrés antes que los ejercicios 8 a 38, los resultados de la tabla 8.12 ya no van a coincidir (por ejemplo el 34 pasa de 5 a otra cantidad si borras pacientes).

---

## 9. Resumen rápido para repasar

- **SQL** = Structured Query Language, lenguaje estándar para bases de datos relacionales. Motores principales: MySQL (el más popular), SQLite (ligero, apps móviles) y MariaDB (fork de MySQL, 100% open source).
- **Tres familias de sentencias:** DDL (estructura: `CREATE · ALTER · DROP · TRUNCATE`), DML (datos: `SELECT · INSERT · UPDATE · DELETE`), DCL (permisos: `GRANT · REVOKE`).
- **Lógica de una consulta:** acción → tabla → condición (`WHERE`). Sin `WHERE`, un `DELETE` borra todo.
- **Cláusulas y operadores:** `FROM` (tabla), `WHERE` (condición), `ORDER BY` (orden); comparación `= > < >= <= <> !=`; lógicos `AND` (todas) / `OR` (una alcanza); palabras clave `BETWEEN · IN · LIKE · IS NULL · IS NOT NULL · NOT`.
- **TRUNCATE vs DELETE:** TRUNCATE es DDL, vacía todo, conserva estructura, resetea identity, no admite `WHERE`; DELETE es DML, admite `WHERE`, no resetea identity.
- **Tipos de datos:** `CHAR` fijo · `VARCHAR` variable · `INT` entero (rango −2.147.483.648 a 2.147.483.647) · `DECIMAL(p,s)` exacto para dinero · `FLOAT` aproximado · `DATE` (`YYYY-MM-DD`) · `BOOLEAN` (0/1) · `ENUM` lista cerrada · `TEXT` largo (hasta 65.535 caracteres, no usable como clave).
- **GROUP BY:** agrupa filas en cajones y devuelve una fila por cajón; toda columna del `SELECT` que no esté en una agregada tiene que estar en el `GROUP BY` (MySQL: error 1055). `WHERE` filtra filas antes de agrupar, `HAVING` filtra grupos después; `COUNT(*)` cuenta filas, `COUNT(col)` ignora `NULL`.
- **Constraints:** `PRIMARY KEY` (única, no repetida, no `NULL`) · `FOREIGN KEY` (relaciona con tabla padre) · `UNIQUE` (no repetida, admite `NULL`) · `NOT NULL` (obligatorio) · `CHECK` (condición) · `DEFAULT` (valor automático). Acciones FK: `CASCADE` (se borra en cadena) · `SET NULL` (queda `NULL`) · `RESTRICT` (no deja borrar).
- **JOIN:** combina filas de dos o más tablas por un campo común. `INNER` = solo coincidencias (intersección) · `LEFT` = todo lo de la izquierda + coincidencias · `RIGHT` = todo lo de la derecha +

coincidencias · `FULL` = todo de ambas, con `NULL` donde no hay coincidencia.

- **Práctica integradora:** ver la sección 8 (`clinica_medica`), que usa todo lo anterior sobre una base real.